How YojanaBench Works — A Super Simple Guide

Imagine you could explain this whole project to a 10-year-old friend.
That is exactly what this document is.

---

1. The Big Idea (in one paragraph)

In India, the government runs thousands of **welfare schemes** — free foodgrains, pensions, scholarships, loans, insurance. Common people often do not get them because:

1. nobody clearly tells them **"Do you qualify?"**,
2. nobody tells them **"How much money will you get?"**, and
3. when the money does **not** arrive in their bank account, nobody can explain **where the money went**.

YojanaBench is a computer program that acts like a **smart, honest helper**. It reads the government's rules, checks a person's application, gives a clear **Yes / No / I need more information** answer with a reason, and — if the money is missing — it **follows the money** like a detective.

> ***One-line summary:*** *A team of small computer helpers (called "agents") that together check scheme eligibility, calculate benefits, and trace missing money — all offline, all deterministic, zero API cost.*

---

2. What is an "AI Agent"?

Think of a **school group project** where each group member has one job:

Group member	Job
The reader	reads the textbook
The checker	checks your answers
The calculator	does the maths
The reporter	writes the final report

In this project, each of those "members" is a small computer program. We call each one an **agent**.

An agent is just a small program that:

1. **Takes in** some information,
2. **Does its one job** with it,
3. **Gives back** a result.

Agents do **not** think or feel like humans. Most of them use plain rules and maths (called **deterministic**), so they always give the *same answer for the same input* — like a calculator, not like a moody person. Only one optional part (a local model called Ollama) can polish the *wording* of an explanation — it never decides the answer and never does the maths.

Why split the work into many agents?

- Each agent has **one job**, so it is easy to test and fix.
- One agent can **double-check** another agent (a second opinion).
- We can **switch off** any agent to study how much it helps (this is called an **ablation study**).

---

3. The Three Parts of the Project

```
YojanaBench
│
├── 1) THE DATASET ──────────── 35 schemes, questions & answers, edge cases
│
├── 2) THE AUDIT PIPELINE ───── 11 agents: "Does this person qualify,
│                               and how much do they get?"
│
└── 3) THE MONEY-TRACE MODULE — 7 agents: "The money never reached me —
                                where did it go?"
```

Plus a **dashboard (website)** to see everything, and a **leaderboard** to compare scores.

---

4. The Three Deliverables Explained Simply

4.1 The Dataset — "The question bank"

- **35 real Indian schemes** (MGNREGA, PM-KISAN style schemes, PDS ration card, scholarships, pensions, crop insurance, …).
- For each scheme we wrote **questions** like:
- *"Is a 70-year-old BPL man eligible?"* → eligibility
- *"How much food grain per month?"* → entitlement
- *"Which papers do I need?"* → documents
- *"Whom do I complain to?"* → appeal
- Each question has a **correct answer written by a human** (ground truth).

- There are also **tricky questions (edge cases)** — like someone with *no age given* — to test if a system honestly says *"I need more info"* instead of guessing.

4.2 The Audit Pipeline — "The inspection team"

Takes one application and produces a **verdict**:

```
ELIGIBLE        → "Yes, you qualify. You get ₹22,100/year."
INELIGIBLE      → "No. Reason: your income is above the limit."
NEEDS_INFO      → "I can't say yet. Please give your age certificate."
```

4.3 The Money-Trace Module — "The money detective"

For the complaint *"I did not get my money"* it finds out whether:

- ✅ the money **arrived**,
- ❌ the government says paid but **nothing arrived**,
- ⚠️ only **part** of the money arrived,
- 🚨 the money went to **someone else's account** (diversion),
- 🔁 the same account is cheating **many people** (a mule ring / fraud gang).

---

5. The AUDIT Pipeline — Meet the 11 Agents

Here is the whole team. Read them like characters in a story.

Step 0 — The Library

📚 Agent 1 — **Corpus Agent** (the Librarian)

Job: Works **before** anything else, only once when the project starts.

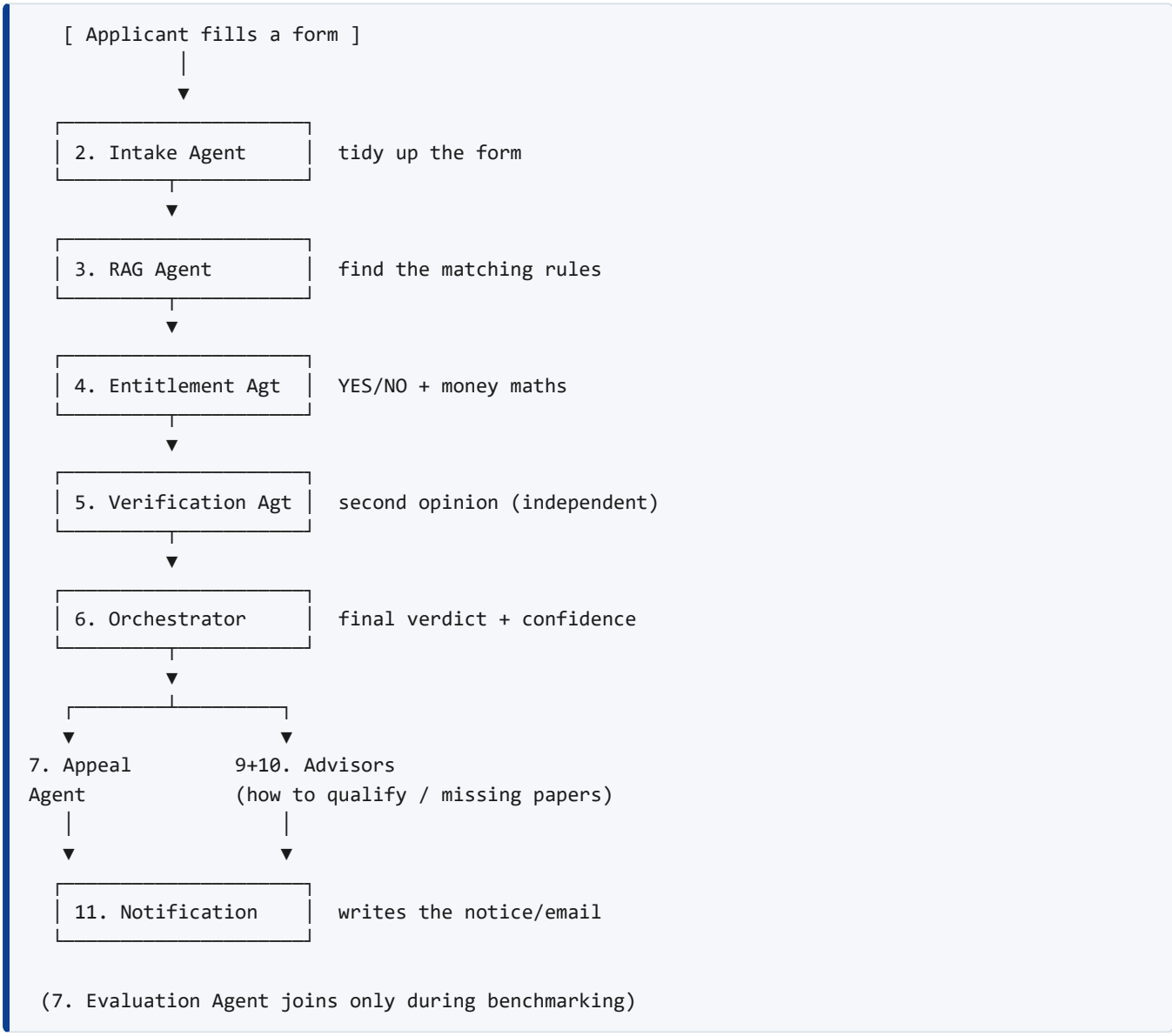
- Reads all 35 scheme files and the guideline documents.
- Cuts them into small pieces called **clauses** (one rule = one piece).
- Builds a **search index** (a card catalogue) so others can find any rule fast.

> ***Kid version:*** *"The librarian arranges all the books on shelves and writes a catalogue, so anyone can find the right page in seconds."*

File: `backend/agents/corpus_agent.py`

The main line (what happens when you press "Check")

Here is the journey of one application, step by step:



📥 Agent 2 — **Intake Agent** (the Receptionist)

Job: Clean up the form before anyone else reads it.

- Fixes field names (e.g. `income` → `annual_income`, `familySize` → `family_size`).
- Converts money written as "2.5" into **₹2,50,000** (lakhs → rupees).
- Turns "yes/true/1" into real true/false values.
- Counts the **documents you submitted** against the list the scheme demands, and writes down **which papers are missing**.
- If a field is blank, it leaves it blank — it **never guesses**.

> ***Kid version:*** *"The receptionist takes your messy form, fixes the spelling, and checks which of your papers are missing."*

File: `backend/agents/intake_agent.py`

🔎 Agent 3 — **RAG Agent** (the Librarian's Assistant / Search Engine)

Job: Find the rules that belong to *this* application.

RAG means **Retrieval-Augmented Generation** — fancy words for **"look up first, then speak."**

- It builds a search query from the scheme name + applicant details.
- It searches the clause index using **two scoring methods**:
- **BM25** — rewards exact keyword matches (like Ctrl+F),
- **TF-IDF cosine** — rewards overlapping important words,
- Both scores are normalised to 0–1 and mixed **50/50**.
- It returns the **top 10 rules** with their IDs and scores — these become the **evidence** that the final verdict must cite.

> ***Kid version:*** *"Before answering, he opens the rulebook and photopies the exact 10 lines that matter, so nobody can say 'you made it up'."*

File: `backend/agents/rag_agent.py` (maths in `backend/retrieval.py`)

⚖️ Agent 4 — **Entitlement Agent** (the Judge + the Cashier)

Job: The main decision maker. It does **two** things.

(a) Eligibility (Yes / No / Need info)

Each scheme has **mandatory rules** (clauses). The agent checks every rule against the applicant using a tiny rule engine:

- every rule gives **True** (passed), **False** (failed), or **Unknown** (info missing);

then:

```
any rule FALSE   → INELIGIBLE
else any UNKNOWN → NEEDS_INFO
else             → ELIGIBLE
```

(b) Money maths (deterministic — never done by an LLM)

Kind of benefit	How it is calculated	Example
Fixed amount	just take the number	Pension = ₹300/month
Per member	rate × family members	5 kg × 4 people = 20 kg
Wage × days	wage × days	₹221 × 100 days = ₹22,100/year
Coverage	the sum-insured number	PMJAY = ₹5,00,000 cover
Tiered	pick the correct slab	income ≤ ₹1 lakh → ₹50,000 loan

The agent also returns a **formula string** like
`"5 kg x 4 members = 20 kg"` so a human can see *how* the number was made.

> ***Kid version:*** *"The judge reads each rule and stamps PASS / FAIL / NEED MORE INFO. The cashier then calculates the amount with real arithmetic — like using a calculator, never guessing."*

File: `backend/agents/entitlement_agent.py`, rules in `backend/condition_eval.py`

🕵️ Agent 5 — **Verification Agent** (the Second Teacher)

Job: Check the first teacher's homework **independently**.

- It re-runs the eligibility check with **its own separate code path**.
- It compares its answer with the Entitlement Agent's answer:

```
both say the same    → agents AGREE
answers differ       → agents DISAGREE → the case MUST go to a human
```

- It also confirms the document checklist.

> ***Kid version:*** *"Two teachers correct your paper separately. If both give the same mark, great. If they disagree, the principal (a human) must look at it."*

File: `backend/agents/verification_agent.py`

🎯 Agent 6 — **Orchestrator** (the Team Leader)

Job: Merge everyone's reports into **one final verdict**.

Decision priority (first match wins):

```
INELIGIBLE  (if eligibility says no)
NEEDS_INFO  (if papers are missing)
NEEDS_INFO  (if the two agents disagreed)
NEEDS_INFO  (if eligibility itself is unsure)
ELIGIBLE    (otherwise)
```

It also computes a **confidence score** from 0.05 to 0.98 by adding small bonuses and one penalty:

```
start at 0.50
+0.20  if we found evidence clauses
+0.15  if both agents agreed
+0.15  if ineligible with a clearly failed rule
+0.10  if eligible with a computed amount
+0.05  if all papers were supplied
-0.20  if the two agents disagreed
```

And the **human-approval gate**:

```
if confidence < 0.60  OR  agents disagree  →  a HUMAN must approve
```

> ***Kid version:*** *"The team leader collects all reports, writes the final decision, and gives it a confidence meter. If the meter is low or his helpers fought each other, he does NOT decide alone — he calls the boss (a human)."*

File: `backend/agents/orchestrator.py`

📣 Agent 7 — **Evaluation Agent** (the School Examiner)

Job: Not part of a single case — it **grades the whole system** against the human-written answer key.

It computes these report-card numbers:

Metric	Simply put
Exact Match	Is the answer word-for-word right? (1 or 0)
Token F1	How many of the *right words* did it say? (0–1)
Verdict Accuracy	How often was Yes/No/Need-info correct? (in %)
Answer Accuracy	How often was the answer "good enough" (F1 ≥ 0.5)?
Entitlement Error	How far off was the money from the true amount?
Document F1	Did it name the right papers?
Calibration ECE	Does its *confidence* match its *actual* accuracy?

Calibration deserves a simple example:
if the agent says *"I'm 90% sure"* ten times, it should be right about **9 times**. If it is right only 4 times, it is over-confident — ECE catches that.

> ***Kid version:*** *"The examiner marks the whole class's answer papers and tells us the average score — and whether the student's confidence is honest."*

File: `backend/agents/evaluation_agent.py`

⚠️ Agent 8 — **Appeal Agent** (the Rights Advisor)

Job: If the verdict is **not** a clean "eligible", tell the person **where to complain and by when**.

- Looks up the scheme's **appeal hierarchy** (level 1 → level 2 → level 3),
- Shows the first authority and its deadline
  (e.g. *"File with the Block Development Officer within 30 days"*),
- **Drafts a ready-to-send appeal letter** for the person.

> ***Kid version:*** *"If they say no, this helper prints the address of the office you should complain to, the last date, and even writes the letter for you."*

File: `backend/agents/appeal_agent.py`

The two Advisory Agents (the special ones)

These two are the project's **novelty** — most scheme finders only say pass/fail; these explain **how to pass**.

🤷 Agent 9 — **Counterfactual Agent** (the "How do I qualify?" Wish Machine)

Job: For an ineligible person, work out the **smallest change** that would flip the answer to Yes.

Steps:

1. Find every rule that failed or is unknown.
2. For each one, compute the required value:
   * income rule "≤ ₹1 lakh" and you have ₹1.5 lakh → *"reduce annual income by ₹50,000"*,
   * age rule "≥ 18" and you are 16 → *"wait 2 years"*.
3. **Prove it:** apply all the suggested changes to a copy of the profile and re-run the real rule engine. If the copy passes → the advice is **verified**, not a guess.
4. If the change is **impossible** (like changing your social category or your age downwards), honestly say: *"No change you can make will help."*
5. Also report which single change alone would be enough.

> ***Kid version:*** *"You failed a level in a game. This helper shows exactly which buttons to change to pass — and then actually TESTS it on a spare copy to be sure the tips really work."*

File: `backend/agents/counterfactual_agent.py`

📋 Agent 10 — **Document Gap Agent** (the Missing-Papers List)

Job: Turn "NEEDS_INFO" into a clear to-do list.

- Lists every missing paper with a **priority number** (1, 2, 3…).
- Tells whether collecting the papers will **actually change** the answer:
- If you are ineligible on income, papers won't help — it says so honestly.
- If you are eligible except for papers → *"Submit all 3 papers and you become eligible."*
- Also lists which **details** (age, income…) are still unknown.

> ***Kid version:*** *"It prints your homework checklist — which paper to bring*

> *first, and whether bringing them will really change your result."*

File: `backend/agents/document_gap_agent.py`

✉️ Agent 11 — **Notification Agent** (the Letter Writer)

Job: Compose the final notice/email containing the decision, the appeal route, and the advice — grounded only in what the other agents found.

Delivery is offline by default: emails/notices are written to an `outbox` folder on disk (a "dry run"). If you configure SMTP settings, they are sent for real.

File: `backend/agents/notification_agent.py`

The Boss of All Agents

🎬 **The Pipeline** (the Director)

File: `backend/pipeline.py`

It does **not** decide anything itself. It simply calls the agents **in order**, times each one, and records a **trace log** like this:

```
IntakeAgent          Normalised 14 fields; 2 missing documents   (0.31 ms)
RAGAgent             Retrieved 10 evidence clauses               (1.02 ms)
EntitlementAgent     Eligibility=eligible; entitlement=22100     (0.15 ms)
VerificationAgent    cross_check=agree; missing_docs=0           (0.08 ms)
Orchestrator         Decision=eligible; confidence=0.9           (0.05 ms)
AppealAgent          not applicable                              (0.02 ms)
CounterfactualAgent  The applicant already satisfies...          (0.04 ms)
DocumentGapAgent     gap=False; verdict_with_docs=eligible       (0.03 ms)
NotificationAgent    approval_notice; actionable=True            (0.11 ms)
```

This log is shown in the dashboard so the working is **transparent and auditable**.

---

6. The MONEY-TRACE Module — Meet the 7 Detectives

The complaint: *"My ₹2,000 pension shows 'paid' in the government records, but my bank account has nothing."*

```
Complaint
   |
   ▼
1. Trace Intake   → tidy the complaint
   ▼
2. DBT Lookup     → find the payment in the government ledger
   ▼
3. Credit Check   → did it land in MY account?
   ▼
4. Reconcile      → compare amounts (₹2000 vs what arrived)
   ▼
5. Fraud Detect   → was it stolen / part of a gang?
   ▼
6. Orchestrator   → final verdict + confidence + what to do
   ▼
7. Grievance Notice → official complaint letter with deadline
```

1. 🔎 **Trace Intake** (the Complaint Clerk)

Takes the complaint, fills in standard names (e.g. `amount` → `expected_amount`), and reports what is **missing** (scheme? UTR number? beneficiary id?). If too much is missing → verdict `needs_info`.

2. 🗄️ **DBT Lookup** (the Records Keeper)

Searches the payment ledger (the list of all government payments) and finds the matching transaction — by **UTR** (a unique payment number, like a tracking number for a courier), or by beneficiary + scheme. It may find: *one match*, *many matches* (ambiguous), or *no match*.

3. 🏦 **Credit Check** (the Bank Spy)

Compares the account number and name on the payment record with the beneficiary's own registered account:

```
right account + name OK     → CREDITED
nothing / zero amount       → NOT CREDITED
money went elsewhere        → DIVERTED   ( 🚨 )
```

It also flags if the payee looks like a **company/agent** instead of a person (words like *pvt, ltd, agency, traders* in the name).

4. 📐 **Reconciliation** (the Accountant)

Plain subtraction and ratio:

```
shortfall   = sanctioned − credited
ratio       = credited / sanctioned
partial     = shortfall > 0
disputed    = |your claim − official sanction| > ₹0.50
```

Example: sanctioned ₹2,000, credited ₹1,500 → shortfall ₹500, ratio 0.75, `partial = true`.

5. 🔍 **Fraud Detect** (the Crime Analyst)

Four checks:

- **Diversion** — money landed in the wrong account.
- **Mule ring** — the same receiving account collects money for **3 or more different people** ⇒ organised fraud gang (`MULE_THRESHOLD = 3`).
- **Duplicate UTR** — same payment reference used more than once.
- **Entity payee** — money routed to a firm, not a person.

Severity: `high` (ring or diversion) → `medium` (any other flag) → `none`.

6. 🎯 **Trace Orchestrator** (the Detective Chief)

Applies the playbook in order and picks the final verdict: `needs_info → not_found → multiple → failed → not_credited → diverted → partial → received`. It attaches a **confidence** (starts at 0.95, drops if information is missing) and a **recommended action** for each verdict.

7. 📜 **Grievance Notice** (the Lawyer)

Writes the official complaint: what happened, the evidence (UTR, sanctioned vs credited amounts, flagged account), the authority to send it to, and the deadline — ready for signature.

---

7. The Ablation Study — "What happens if we switch one helper off?"

To **prove** each agent earns its place, the same benchmark is run in four ways:

Variant	What is changed	What we expect
`full`	everything on	best scores
`no_verification`	2nd opinion removed	more errors slip through
`llm_math`	calculator replaced by "grab the first number"	big entitlement errors
`no_counterfactuals`	advice agents not scored	edge-case accuracy drops

Each variant's score is compared with `full` as a difference in **percentage points**. This is how the project shows, with numbers, that (1) the cross-check catches mistakes and (2) deterministic maths prevents wrong money calculations.

File: `backend/ablation.py`

---

8. The Leaderboard — "Who scored highest?"

Anyone can run their own model through the **same** answer key:

```
python benchmark.py --name "MyModel" --predictions my_predictions.json
```

Entries are ranked by three keys, in order:

1. higher **verdict accuracy**,
2. then higher **answer accuracy**,
3. then **lower** calibration error (better honesty).

File: `backend/leaderboard.py`

---

9. The Dashboard (the Website)

A single web page (served by FastAPI at `http://127.0.0.1:8000`) where a user can:

- log in as a **beneficiary** or an **official** (role-based login),
- fill an application form and see the verdict, evidence clauses, formula and confidence,
- see the **agent trace** (which helper ran, what it said, how many ms),
- run the **benchmark** and see accuracy tables and charts,
- run the **ablation** and see the four variant scores,
- file a **"money not received" complaint** and watch the 7 detectives work,
- edit email/SMS settings.

---

10. Key Properties (why this design is trustworthy)

1. **Deterministic** — same input, always the same output. No guessing.
2. **Offline & free** — no paid APIs. The optional local LLM only rewords explanations.
3. **Tri-state honesty** — it can say **"I don't know, give me more info"** instead of forcing a Yes/No.
4. **Citeable** — every verdict lists the clause IDs it used.
5. **Cross-checked** — two independent agents must agree, else a human decides.
6. **Proven advice** — counterfactual suggestions are re-tested before being shown.

7. **Auditable** — every agent's step is logged with timing.

---

11. One-Page Summary (memorise this)

```
THE QUESTION:  "Do I qualify for this government scheme, how much do I get,
                and where is my money?"

THE ANSWER:    A team of 11 small rule-based agents (audit) + 7 detectives
               (money trace) working in a fixed order.

  Librarian → Receptionist → Searcher → Judge/Cashier → Second Teacher
           → Team Leader → Examiner → Rights Advisor → Wish Machine
           → Missing-Papers List → Letter Writer

  Complaint → Clerk → Records Keeper → Bank Spy → Accountant
           → Crime Analyst → Detective Chief → Lawyer

THE GUARANTEES: deterministic maths, tri-state verdicts, cited evidence,
                dual-agent agreement, human gate below 0.60 confidence,
                fully offline, zero API cost.
```

---

12. Where Each Piece Lives in the Code

What	File
Agent team director	`backend/pipeline.py`
Librarian (corpus)	`backend/agents/corpus_agent.py`
Receptionist (intake)	`backend/agents/intake_agent.py`
Searcher (RAG)	`backend/agents/rag_agent.py`
Judge + Cashier (entitlement)	`backend/agents/entitlement_agent.py`
Rule engine (pass/fail logic)	`backend/condition_eval.py`
Search maths (BM25 / TF-IDF)	`backend/retrieval.py`
Second teacher (verification)	`backend/agents/verification_agent.py`
Team leader (orchestrator)	`backend/agents/orchestrator.py`
Examiner (metrics)	`backend/agents/evaluation_agent.py`
Rights advisor (appeal)	`backend/agents/appeal_agent.py`
Wish machine (counterfactual)	`backend/agents/counterfactual_agent.py`
Missing-papers list	`backend/agents/document_gap_agent.py`
Letter writer (notice)	`backend/agents/notification_agent.py`
Ablation study	`backend/ablation.py`
Leaderboard	`backend/leaderboard.py`
Money detectives	`backend/trace/*.py`
Scheme data (35 schemes)	`data/schemes/*.json`
Questions & answers	`data/qa/qa_benchmark.json`
Mathematical formulas	`docs/Mathematical_Calculations.md`

Written in simple language on purpose — if a kid can read this, the project is explained well.